

Context API в React

Введение в Context API

Context API — это механизм в React, который позволяет передавать данные через дерево компонентов без необходимости передавать props вручную на каждом уровне. Context спроектирован для передачи данных, которые можно считать «глобальными» для дерева компонентов React, например, текущий аутентифицированный пользователь, тема оформления или предпочитаемый язык.

Когда использовать Context

Context в основном применяется, когда некоторые данные должны быть доступны многим компонентам на разных уровнях вложенности. Используйте его с осторожностью, поскольку это усложняет повторное использование компонентов.

Если вы просто хотите избежать передачи некоторых props через множество уровней, композиция компонентов часто является более простым решением, чем Context.

API Context

React.createContext

```
const MyContext = React.createContext(defaultValue);
```

Создаёт объект Context. Когда React рендерит компонент, который подписывается на этот объект Context, он будет читать текущее значение контекста из ближайшего соответствующего Provider выше в дереве компонентов.

Аргумент `defaultValue` используется только когда компонент не имеет соответствующего Provider выше в дереве. Это может быть полезно для тестирования компонентов в изоляции без необходимости оборачивать их.

Context.Provider

```
<MyContext.Provider value={/* некоторое значение */}>
```

Каждый объект Context имеет компонент Provider, который позволяет компонентам-потребителям подписываться на изменения контекста.

Принимает проп `value`, который будет передан компонентам-потребителям этого контекста, которые являются потомками этого Provider. Один Provider может быть связан со многими

потребителями. Провайдеры могут быть вложены друг в друга, переопределяя значения контекста глубже в дереве.

Context.Consumer

```
<MyContext.Consumer>
  {value => /* отрендерить что-то на основе значения контекста */}
</MyContext.Consumer>
```

Компонент React, который подписывается на изменения контекста. Это позволяет вам подписаться на контекст в функциональном компоненте.

Требует функцию в качестве дочернего элемента. Функция получает текущее значение контекста и возвращает узел React. Аргумент **value**, переданный функции, будет равен проп **value** ближайшего Provider для этого контекста выше в дереве. Если нет Provider для этого контекста выше, аргумент **value** будет равен **defaultValue**, который был передан в **createContext()**.

useContext

```
const value = useContext(MyContext);
```

Хук **useContext** принимает объект контекста (возвращаемый из **React.createContext**) и возвращает текущее значение контекста для этого контекста. Текущее значение контекста определяется пропом **value** ближайшего **<MyContext.Provider>** выше вызывающего компонента в дереве.

Когда ближайший **<MyContext.Provider>** выше обновляется, этот хук вызовет повторный рендер с последним значением контекста, переданным этому провайдеру контекста.

Примеры использования Context API

Простой пример с темой оформления

```
import React, { createContext, useContext, useState } from 'react';

// Создаем контекст с значением по умолчанию 'light'
const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('light');

  return (
    <ThemeContext.Provider value={theme}>
      <div>
        <Header />
      </div>
    </ThemeContext.Provider>
  );
}
```

```

        <Main />
        <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
            Переключить тему
        </button>
    </div>
</ThemeProvider>
);
}

function Header() {
    const theme = useContext(ThemeContext);
    return (
        <header className={`header-${theme}`}>
            <h1>Заголовок с темой {theme}</h1>
        </header>
    );
}

function Main() {
    return (
        <main>
            <Content />
        </main>
    );
}

function Content() {
    const theme = useContext(ThemeContext);
    return (
        <section className={`content-${theme}`}>
            <p>Содержимое с темой {theme}</p>
        </section>
    );
}

```

Пример с несколькими контекстами

```

import React, { createContext, useContext, useState } from 'react';

const ThemeContext = createContext('light');
const UserContext = createContext({ name: 'Гость' });

function App() {
    const [theme, setTheme] = useState('light');
    const [user, setUser] = useState({ name: 'Гость' });

    const login = () => {
        setUser({ name: 'Пользователь' });
    };
}

```

```

return (
  <ThemeContext.Provider value={theme}>
    <UserContext.Provider value={{ user, login }}>
      <Layout />
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
        Переключить тему
      </button>
    </UserContext.Provider>
  </ThemeContext.Provider>
);
}

function Layout() {
  return (
    <div>
      <Header />
      <Content />
    </div>
  );
}

function Header() {
  const theme = useContext(ThemeContext);
  const { user } = useContext(UserContext);
  return (
    <header className={`header-${theme}`}>
      <h1>Привет, {user.name}!</h1>
    </header>
  );
}

function Content() {
  const theme = useContext(ThemeContext);
  const { user, login } = useContext(UserContext);
  return (
    <section className={`content-${theme}`}>
      <p>Текущий пользователь: {user.name}</p>
      {user.name === 'Гость' && (
        <button onClick={login}>Войти</button>
      )}
    </section>
  );
}

```

Пример с обновлением контекста

```

import React, { createContext, useContext, useReducer } from 'react';

// Создаем контекст
const CounterContext = createContext();

```

```

// Редьюсер для управления состоянием счетчика
function counterReducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error(`Неизвестное действие: ${action.type}`);
  }
}

// Провайдер контекста, который предоставляет состояние и функцию dispatch
function CounterProvider({ children }) {
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });

  return (
    <CounterContext.Provider value={{ state, dispatch }}>
      {children}
    </CounterContext.Provider>
  );
}

// Хук для использования контекста счетчика
function useCounter() {
  const context = useContext(CounterContext);
  if (context === undefined) {
    throw new Error('useCounter должен использоваться внутри CounterProvider');
  }
  return context;
}

// Компонент, который отображает и обновляет счетчик
function Counter() {
  const { state, dispatch } = useCounter();
  return (
    <div>
      <p>Счетчик: {state.count}</p>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
    </div>
  );
}

// Приложение, которое использует провайдер контекста
function App() {
  return (
    <CounterProvider>
      <div>
        <h1>Пример с Context API и useReducer</h1>
        <Counter />
      </div>
    </CounterProvider>
  );
}

```

```
    </CounterProvider>
  );
}
```

Оптимизация производительности

Поскольку контекст использует сравнение по ссылке для определения, когда нужно перерендерить, есть некоторые подводные камни, которые могут вызвать непреднамеренные рендеры в потребителях, когда родительский компонент перерендеривается.

Разделение контекстов

Один из способов оптимизации — разделить контексты, которые не всегда изменяются вместе.

```
const ThemeContext = createContext('light');
const UserContext = createContext({ name: 'Гость' });

function App() {
  const [theme, setTheme] = useState('light');
  const [user, setUser] = useState({ name: 'Гость' });

  // Теперь изменение темы не вызовет перерендер компонентов,
  // которые используют только UserContext
  return (
    <ThemeContext.Provider value={theme}>
      <UserContext.Provider value={user}>
        <Layout />
      </UserContext.Provider>
    </ThemeContext.Provider>
  );
}
```

Мемоизация значения контекста

Другой способ оптимизации — мемоизировать значение контекста с помощью `useMemo`.

```
function App() {
  const [theme, setTheme] = useState('light');
  const [user, setUser] = useState({ name: 'Гость' });

  // Мемоизируем объект value, чтобы он не создавался заново при каждом рендере
  const userContextValue = useMemo(() => ({ user, setUser }), [user]);

  return (
    <ThemeContext.Provider value={theme}>
      <UserContext.Provider value={userContextValue}>
        <Layout />
      </UserContext.Provider>
    </ThemeContext.Provider>
  );
}
```

```
    </ThemeContext.Provider>  
  );  
}
```

Заключение

Context API предоставляет мощный способ передачи данных глубоко в дерево компонентов без необходимости передавать props через промежуточные компоненты. Это особенно полезно для глобальных данных, таких как тема оформления, аутентифицированный пользователь или языковые настройки.

Однако, Context не следует использовать для каждого случая передачи данных, так как это может усложнить повторное использование компонентов. Для многих случаев композиция компонентов является более подходящим решением.

При использовании Context важно помнить о возможных проблемах с производительностью и применять соответствующие оптимизации, такие как разделение контекстов и мемоизация значений контекста.